

```

def getBalance(accountNumber, password):
    ❸ global account0Name, account0Balance, account0Password
    global account1Name, account1Balance, account1Password

    if accountNumber == 0:
        if password != account0Password:
            print('Incorrect password')
            return None
        return account0Balance
    if accountNumber == 1:
        if password != account1Password:
            print('Incorrect password')
            return None
        return account1Balance

--- snipped additional deposit() and withdraw() functions ---

--- snipped main code that calls functions above ---

print('Done')

```

Listing 1-4: Bank simulation for two accounts with functions

Even with just two accounts, you can see that this approach gets out of hand quickly. First, we set three global variables for each account at ❶, ❷, and ❸. Also, every function now has an `if` statement to choose which set of global variables to access or change. Any time we want to add another account, we'll need to add another set of global variables and more `if` statements in every function. This is simply not a feasible approach. We need a different way to handle an arbitrary number of accounts.

Implementation 4—Multiple Accounts Using Lists

To more easily accommodate multiple accounts, in [Listing 1-5](#) I'll represent the data using lists. I'll use three parallel lists in this version of the program: `accountNamesList`, `accountPasswordsList`, and `accountBalancesList`.

File: Bank4_N_Accounts.py

```

# Non-OOP Bank
# Version 4

```